# REVIEW

# Independent §11.4.142/§11.4.125 code-review — bug-class + parser fixes

**Reviewer role:** BACKGROUND operator-authorized independent review, READ-ONLY NON-COMMITTING. **Did NOT** edit / stage / `git add` / commit anything. Ran only read-only `go test` / `go build` / `go vet` (artifacts gitignored, not staged). **Revision:** 1 **Last modified:** 2026-06-16T16:27:12Z **Scope:** `4fbe581` (Translate-arg data-loss, 6 sites) + `7cabe3f` (MINOR-W6-1 parser title-dup). **Discipline:** §11.4.142 + §11.4.1 + §11.4.6 (FACT-only, evidence-backed) + §1.1 (mutation) + §11.4.69.

---

## VERDICT

| Fix | Verdict |
|---|---|
| `4fbe581` Translate-arg data-loss (Seam A/B/C, 6 sites) | **GO** |
| `7cabe3f` MINOR-W6-1 parser title-dup | **GO** |

Both fixes are release-ready. No findings rise to BLOCKING. Two NON-BLOCKING observations (one nit, one accuracy note on the evidence doc's wording) recorded below — neither affects correctness or releasability.

---

## FIX 1 — `4fbe581` Translate-arg data-loss — GO

### 1.1 The choke-point claim is TRUE (independently traced) — §11.4.6

The wire contract was independently re-verified, not taken on the commit's word:

- **`OpenAIClient.Translate(ctx, text, prompt)` sends `prompt` (arg-2) as the single user message and ignores `text` (arg-1)** — confirmed at `pkg/translator/llm/openai.go` (`Messages: [{Role:"user", Content: userMessage}]`, `userMessage` derived from `prompt` with the new fallback).
- **`bridge.clientTranslator.client` is the raw OpenAI-compatible `llmClient`** (`pkg/bridge/bridge.go:546`), and the package's own convention

`realInvokeDispatch` already calls `client.Translate(ctx, "", full)` (`bridge.go:369`). Seam B mirrors this exactly.

**6-site trace to a choke-point (all reach one):**

| # | site | actual content shape | type at runtime | reaches choke-point? |
|---|---|---|---|---|
| 1 | `cmd/markdown-translator/main.go:244` | content in arg-1, `""` arg-2 → raw `bridge.BestClient` | raw `OpenAIClient` | **Seam A fallback + Seam C real prompt** ✓ |
| 2 | `pkg/preparation/coordinator.go:290` | `Translate(ctx, prompt, "")` | `*LLMTranslator` (built-in) OR `clientTranslator` (ensemble) | see note ‡ |
| 3 | `pkg/preparation/coordinator.go:361` | same | same | see note ‡ |
| 4 | `pkg/preparation/coordinator.go:424` | same | same | see note ‡ |
| 5 | `pkg/verification/polisher.go:563` | `Translate(ctx, prompt, location)` | `*LLMTranslator` OR `clientTranslator` | **Seam B compose** (ensemble path) ✓ |
| 8 | `pkg/coordination/multi_llm.go:445` | `Translate(ctx, text, contextHint)` | `*LLMTranslator` OR `clientTranslator` | **Seam B compose** ✓ |
| 9 | `pkg/coordination/multi_llm.go:529` | same | same | **Seam B compose** ✓ |

‡ **Accuracy note (NON-BLOCKING) on the evidence doc's "EMPTY-PAYLOAD" labelling of #2/#3/#4.** Independently traced: `*LLMTranslator.Translate` (`pkg/translator/llm/llm.go`) does NOT forward the caller's args verbatim — it builds its own non-empty prompt via `createTranslationPrompt(text, contextStr)` and calls `lt.client.Translate(ctx, text, prompt)` with that **always-non-empty** prompt in arg-2. Therefore, **when the preparation provider is the built-in `*LLMTranslator`, sites #2/#3/#4 were NOT broken** (arg-2 was never empty; Seam A's fallback is inert there). They are only the "empty-payload" class **when an ensemble factory injects a raw/bridge client** — and in that case Seam A (raw `OpenAIClient`) or Seam B (`clientTranslator`, where `composeForClient` with empty context sends the prompt verbatim — proven by `TestClientTranslator_EmptyContextSendsContentVerbatim`) covers them.
→ The fix is correct for every routing of #2/#3/#4; the only imprecision is the evidence doc calling them unconditionally "broken/empty-payload." This is a documentation-wording nit, **not** a code defect.

**"Leave callers unchanged" claim — TRUE and CORRECT.** Rewriting the preparation/polisher callers to `("", prompt)` would (a) make `clientTranslator` refuse on empty content and (b) make `*LLMTranslator` early-return on empty `text` (`if text=="" ... return text,nil`) — breaking both paths. Leaving them as `(prompt, "")` / `(prompt, location)` and fixing at the choke-points is the right call.

## 1.2 RED tests are httptest-real and catch the negation — §11.4.27 / §1.1 / §11.4.115

Neither test uses `MockLLMClient`. Both stand up an `httptest.Server` and a **real `OpenAIClient`** / **real `clientTranslator`**, asserting the actual wire user-message content.

Mutation proof executed this session (RED_MODE=1 on the FIXED code MUST FAIL):

```
GREEN default:  ok pkg/translator/llm ; ok pkg/bridge
RED_MODE=1 TestOpenAITranslate_EmptyPayloadGuard      -> FAIL
("OK::The old man..." not boilerplate)  ✓ catches negation
RED_MODE=1 TestClientTranslator_ComposesContentNotLabel -> FAIL
(compose sends real content)            ✓ catches negation
```

The assertions are load-bearing, not tautologies. §11.4.69 refuse-empty paths covered by `TestOpenAITranslate_RefuseAllEmpty` + `TestClientTranslator_RefusesEmptyContent` (assert no request reaches the provider on empty content).

## 1.3 Seam B compose is correct — content + label both reach the model

`composeForClient(text, contextStr)`:

- empty context → `text` verbatim (no double-wrap) ✓
- non-empty context → `text + "\n\nContext: " + contextStr` ✓ (content first, label as labelled hint; never substituted, never dropped) No double-send / malformation. Verified on the wire by the GREEN assertions (`Contains(captured, realContent)` AND `Contains(captured, label)`).

## 1.4 No SAFE site regressed

- `batch_handlers.go:146` `(req.Text, "")`: via `*LLMTranslator`, empty input early-returns before any client call; non-empty → content sent verbatim. No new error on a legitimate path.
- `distributed/coordinator.go:308` `(text, contextHint)`: if `clientTranslator`, Seam B now composes correctly — an **improvement**, not a regression.
- `markdown/simple_workflow.go:92` `(text, prompt)`: prompt non-empty → Seam A inert; content reaches via prompt.
- `TestOpenAITranslate_RequestShapeAndAuth` (asserts non-empty arg-2 reaches the wire verbatim) — **still green** (re-run this session). The Seam-A fallback is provably inert for non-empty arg-2.

- The new refuse-empty (Seam A + Seam B) only fires on empty/whitespace **content**, which has no legitimate translation; multi_llm treats it as a failed attempt (`err==nil && translated!=""` gate) and retries/exhausts — previously it would store provider boilerplate. Behaviour change is strictly an improvement.

### 1.5 Validation (this session, read-only)

```
go build ./...  -> exit 0
go vet ./pkg/translator/llm/ ./pkg/bridge/ ... -> exit 0
go test ./pkg/translator/llm/ ./pkg/bridge/ ./pkg/coordination/ \
        ./pkg/verification/ ./pkg/preparation/ \
        ./cmd/markdown-translator/ ./pkg/markdown/ -count=1 -> ALL
ok
```

---

## FIX 2 — `7cabe3f` MINOR-W6-1 parser title-dup — GO

### 2.1 Root cause + fix correct

- `html_parser.go`: `extractTextWithContext` now skips `<head>` (was leaking `<title>` into Content) + `stripLeadingTitle` drops a leading body `<h1>`==title.
- `epub_parser.go`: `stripLeadingTitle` drops the residual leading `<h1>`==title (`<head>` already stripped).
- Title now carried EXACTLY ONCE (in `chapter.Title`); `bookToString` prepends it once.

### 2.2 `stripLeadingTitle` is a guarded no-op — no over-strip

- empty/whitespace title → no-op ✓
- only acts on the START of left-trimmed content (`HasPrefix`); interior repeats preserved ✓
- **word-boundary guard**: char after the title must be whitespace or EOC, so `"Cat"` does NOT strip `"Caterpillar"` ✓ (directly table-tested in `title_duplication_bughunt_test.go`).
- multibyte titles: a non-space continuation byte fails the `rest[0]` whitespace check → safe no-op.
- It does NOT strip a legitimate mid-content body line equal to a short title (leading-only). ✓ Minor: it collapses leading blank lines after the removed title line — acceptable (the title line itself is being removed).

### 2.3 Tests httptest-/parser-real and catch the negation — §1.1

No mocks: `NewHTMLParser().Parse` / `NewEPUBParser().Parse` / `bookToString` driven directly; plus a `stripLeadingTitle` edge-case table. Mutation proof executed this session:

```
GREEN default: ok pkg/ebook ; ok cmd/unified-translator
RED_MODE=1 TestBugHunt_HTMLTitleDuplication -> FAIL ("got 0" leaks
on fixed)  ✓ catches negation
RED_MODE=1 TestBugHunt_EPUBTitleDuplication ->
FAIL                            ✓ catches negation
```

The roundtrip guard asserts body text survives AND title count == 1.

### 2.4 Validation (this session, read-only)

```
go test ./pkg/ebook/ ./cmd/unified-translator/ -count=1 -> ok ok
  (incl. the pre-existing epub_head_multiline_bughunt regression —
green)
go vet (pkg/ebook, cmd/unified-translator) -> exit 0
```

---

# Findings summary

- **NO BLOCKING findings.** Both fixes are correct, evidence-backed, mutation-proven, and break no SAFE site.
- **NON-BLOCKING (doc-wording, §11.4.6 precision):** the `4fbe581` evidence doc / commit calls preparation sites #2/#3/#4 unconditionally "EMPTY-PAYLOAD broken." Independently, they are broken ONLY when routed through a raw/bridge client; via the built-in `*LLMTranslator` they were already correct (arg-2 always non-empty). The CODE handles both routings correctly — this is purely an over-broad description, not a defect. No commit required for the GO; flagged for accuracy if the doc is later revised.
- **NON-BLOCKING (nit):** `stripLeadingTitle` collapses leading blank lines following the removed title — cosmetically fine.

# Release-readiness

Both `4fbe581` and `7cabe3f` are **release-ready**. The data-loss class is genuinely closed at the two choke-points (real-content reaches the model; empty content fails loudly per §11.4.69), and the title-dup class is closed at the parser layer with exactly-once title carriage. Standing §11.4.135 guards are real (httptest/parser-driven, negation-catching) and registered with RED_MODE polarity.